PSSA: A Precise Static Analysis Framework for Detecting Vulnerabilities in CMS Plugins

Zhongfu Su, Cong Wu, Xi Tan, Jing Chen, Ruiying Du, Yang Liu, *Senior Member, IEEE*, and Yang Xiang, *Fellow, IEEE*

***Abstract*—Content management systems (CMS) have been the preferred option for rapidly developing web applications due to their convenience. Their extensive plugin ecosystems also allow for quick and easy expansion of web application functionality. With the increasing complexity of plugins, there has been a surge in plugin vulnerabilities, which pose a serious threat to the overall security of web applications. Current methods for analyzing plugins simply apply typical web application analysis techniques, which fail to consider the unique characteristics of plugins and lead to inaccuracy. This paper presents PSSA, a novel method for detecting web vulnerabilities in PHP-based CMS plugins. The proposed approach incorporates an in-depth analysis of the CMS framework's context during plugin analysis and enhances the analysis of PHP object-oriented code to achieve precise vulnerability detection. To evaluate its effectiveness, we compare PSSA with existing tools for vulnerability detection using established vulnerability datasets. Our results demonstrate that PSSA outperforms other tools, detecting the highest number of vulnerabilities while minimizing false positives. Additionally, we apply PSSA in a comprehensive survey of popular WordPress plugins, evaluating 980 extensively used plugins. This thorough investigation brings to light 178 new vulnerabilities, 124 of which are found in plugins boasting over 1 million downloads. Our efforts also contribute to the acknowledgment of 82 CVE identifiers, further underscoring the impact and necessity of our research in enhancing CMS plugin security.**

***Index Terms*—Vulnerability detection, static analysis, content management systems.**

I. Introduction

As WEB application development becomes increasingly complex, many developers turn to content management systems (CMS) to contribute, schedule or manage web content to be published. In addition, CMS allows plugins to extend the core functionality of CMS framework. The wide variety of available plugins has contributed to the increasing popularity of CMS [1], [2]. W3Techs statistics reveal that 80.4% of current websites employ CMS, with WordPress being the most prominent, powering 62.0% of all CMS-based websites [3]. And the official WordPress plugin marketplace houses over 59,000 plugins [4].

Given that the majority of CMS frameworks make use of the PHP server-side language [3], we focus on PHP-based CMS platforms. The CMS framework uses dynamic function invocation and reflection to call plugin code. And CMS plugins run within the environment of the CMS framework, extensively utilizing global resources and helper functions provided by the CMS framework. Thus, vulnerabilities in plugins can have the same impact as vulnerabilities in the framework itself. According to WPScan [5], only 1% of WordPress-related vulnerabilities are due to WordPress itself, whereas plugins account for 95% of such vulnerabilities.

Previous research has studied the security of CMS frameworks [6], [7] and plugins, including the detection of malicious behavior [1] and privacy violations [8]. However, most existing techniques focus on generic vulnerability detection methods for CMS frameworks and do not account for the unique execution context of plugins [9]. As a result, they fail to achieve accurate plugin-level analysis. For example, tools such as phpSAFE [9] and RIPS [6] are unable to recognize that WordPress helper functions like `add_query_arg` can return tainted values. This omission causes critical taint flows to be missed, leading to both false positives and false negatives during vulnerability detection.

Modern CMS platforms and their plugins are predominantly implemented using object-oriented programming (OOP) paradigms. However, static analysis of OOP constructs in PHP is particularly challenging due to the language's weak typing and dynamic type changes at runtime [6], [10]. Existing tools such as RIPS [6] and phpSAFE [9] largely approximate method calls as ordinary function calls, leading to imprecise call resolution and inaccurate taint propagation. Tchecker [7] improves call graph construction by iteratively inferring variable types through inter-procedural analysis, yet its object-insensitive design and partially inter-procedural taint tracking limit its ability to handle dynamic calls and string manipulations, resulting in both false negatives and false positives. Moreover, for scalability reasons, most prior approaches adopt path-insensitive analysis strategies [6], [7], which fail to recognize path-sensitive sanitization

Received 5 October 2025; revised 2 June 2026; accepted 30 June 2026. Date of publication 13 July 2026; date of current version 1 September 2026. This work was supported in part by the National Key R&D Program of China under Grant2021YFB2700200, in part by the National Natural Science Foundation of China under Grant 62076187, and in part by the Key R&D Program of Hubei Province under Grant2021BAA190 and Grant 2022BAA039. Recommended for acceptance by Jun Dai. *(Corresponding author: Cong Wu.)*

Zhongfu Su, Cong Wu, Jing Chen, and Ruiying Du are with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: jrxnm666@gmail.com; cnacwu@whu.edu.cn; chenjing@whu.edu.cn; duruiying@whu.edu.cn).

Xi Tan is with the Department of Computer Science, University of Colorado Colorado Springs(UCCS), Colorado Springs CO 80918 USA (e-mail: xtan4@uccs.edu).

Yang Liu is with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: yangliu@ntu.edu.sg).

Yang Xiang is with the Digital Capability Research Platform, Swinburne University of Technology, Melbourne VIC 3122, Australia (e-mail: yxiang@swin.edu.au).

Digital Object Identifier 10.1109/TDSC.2026.3712242

and validation logic. Our empirical analysis shows that such path-sensitive security checks are pervasive in real-world PHP CMS plugins, and ignoring them substantially inflates false positives. Given the rapid growth of plugin vulnerabilities in recent years [5], there is a clear need for analysis techniques that integrate CMS-specific context, accurately model object-oriented PHP code, and selectively account for path-sensitive behaviors.

To address those limitations, we introduce PSSA (Plugin Static Security Analyzer), an effective approach for the precise static security analysis of PHP-based CMS plugins. PSSA comprises two components: the CMS Knowledge Collector and the Static Analyzer. In the first stage, PSSA gathers all CMS-related knowledge that may impact plugin analysis through static analysis and dynamic testing, including global variables and helper functions. Subsequently, PSSA performs a summary-based, context-sensitive, interprocedural dataflow analysis for taint-style vulnerability detection. Then PSSA conducts a type inference analysis to determine the types for each variable, propagating them during dataflow analysis. To minimize false positives caused by path-insensitive analysis, PSSA performs vulnerability-oriented, path-sensitive analysis for any tainted variable. PSSA is implemented and evaluated via performing common SQL injection and XSS vulnerability analyses for WordPress plugins. Our evaluation results show that PSSA can identify new vulnerabilities with few false positives. To systematically address the above challenges, we made the following contributions when building PSSA.

- We propose a scalable static analysis approach for PHP-based CMS plugin vulnerability detection. By integrating CMS framework environments into our analysis process, we enhance the accuracy and efficacy of the static analysis, enabling a more robust evaluation of potential vulnerabilities.
- We propose a summary-based vulnerability-oriented path-sensitive analysis for PHP object-oriented code to detect more vulnerabilities and contain fewer false positives.
- We implemented PSSA to target WordPress plugins, conducting extensive tests on popular plugins from the official WordPress repositories. Finally, we identify 178 new vulnerabilities and contribute 82 new CVEs using PSSA. Comparative evaluations with existing state-of-the-art tools demonstrate that PSSA not only uncovers more vulnerabilities but also significantly reduces false positives in detecting vulnerabilities.

II. Background and Motivating Example

A. Static Analysis on PHP Code

Summary-based static analysis for PHP was proposed by Xie [11] and extended by RIPS [6]. It abstracts data flow information into block summaries and function summaries, thereby achieving context-sensitive data flow analysis.

Symbols: Symbols serve as an intermediate representation for memory locations. For instance, `Variable` represents a `$variable` by its name, while `UnknowTypeVariable` represents a variable with an unidentifiable type, such as function parameters and global variables. `Value` signifies a static value extracted from the source code.

Block summary: A block summary records the outcomes of a basic block, focusing on data flow and symbol restrictions, leveraged for static taint analysis and the construction of function summaries [11]. PSSA extends prior methodologies, incorporating the following properties:

- *DataFlow:* Record the symbols corresponding to the variables visible in this block.
- Constants. Store all constants defined in this block.
- *IsExitBlock:* Indicate if the block is an exit block.
- *ReturnValue:* Store the return symbol defined in this block we inferred before.
- *PropWriteCache:* Record the assignment of the property of known and unknown object symbols in this block.

Data flow summary information in the parent block will propagate to the child block. Because the analysis is path-insensitive, data flow information is merged when multiple blocks converge.

Function summary: Function summaries encapsulate interprocedural information for analysis. From a function's Control Flow Graph (CFG), basic block summaries are initially constructed. This is followed by a path-insensitive, intra-procedural data-flow analysis to infer function summaries. Blocks without subsequent nodes signify the function's end, return, or exit points. Key to data flow analysis in function summaries are two primary fields:

- *ReturnValue:* A set of all possible return value symbols.
- *IsExitFunction:* The tag is set if a function exits in any path.

Previous summary-based, interprocedural static analysis methods for PHP struggled with object-oriented code and failed to recognize path-sensitive security measures due to their path-insensitive nature. This paper enhances these methods to support object-oriented code analysis. It also identifies common path-sensitive security measures.

B. Analysis of CMS Framework

Modern CMS frameworks typically interact with plugins through dynamic features such as reflection and hooks. Conversely, plugins can also access global objects, functions, and methods within the CMS framework. Performing precise static or dynamic analysis on these dynamic features in a scripting language like PHP is particularly challenging [6], [7], [12]. Consequently, detailed modeling of the relevant components within the CMS framework that influence plugin behavior is essential.

Previous research, including our own, has attempted to handle various dynamic features in PHP, such as dynamic function calls and dynamic includes [6], [7]. While these approaches offer some utility for common vulnerability analysis, they fall short of providing a holistic understanding of the entire CMS's impact on plugins. Furthermore, some plugin analysis works neglect this influence entirely [9].

Our work aims to model the aspects of complex CMSs that affect plugin vulnerability analysis and automate their preprocessing extraction. This approach consequently enhances the

```
1  <?php
2  //Path-sensitive Sanitization
3  if(!is_numeric($var)){
4    // $var = 0;
5    // unset($var);
6    $var = addslashes($var);
7  }
8  $var = is_numeric($var)?$var:0;
9
10 //Path-sensitive Termination
11 if(!is_numeric($var)){
12   // die(0);
13   return;
14 }
15
16 //Path-sensitive Validation
17 if(!is_numeric($var)){
18   $error = true;
19 }
20 if(!$error){
21   ...
22 }
23 ... ?>
```

Listing 1: Example for path-sensitive security mechanisms.

efficiency, precision, and recall rates of subsequent plugin vulnerability analysis.

C. Path-Sensitive Sanitization and Validation

Earlier research [13] identified common security mechanisms in PHP web applications, such as generic and context-sensitive input sanitization and validation. Despite some exploration of these mechanisms [6], path-sensitive security measures are less addressed in static analysis. These can be broadly categorized into three types.

- *Path-sensitive sanitization:* Variables are sanitized based on conditional results. For instance, in Listing 1, a non-numeric `$var` is sanitized for security (lines 3-7).
- *Path-sensitive termination:* The code exits if certain conditions are met or not met. In Listing 1, the code terminates if `$var` is non-numeric (lines 11-14).
- *Path-sensitive validation:* Validates input based on conditions. In Listing 1, validation on line 17 affects the code flow to line 20 via the `$error` variable (lines 17-22).

Previous studies have shown the difficulty in statically recognizing path-sensitive security mechanisms in code analysis tools [13]. Static analysis tools often opt for path-insensitive analysis to enhance performance, analyzing each execution path independently. This approach fails to detect actions like sanitization or termination within branches, potentially leading to large numbers of false positives. To overcome this, we simulate block edges for each user-controlled variable and function parameter, allowing for the collection of restrictions which stored in block summary. By propagating the restrictions of each symbol, our method more effectively handles path-sensitive sanitization and validation.

D. Motivating Example

WordPress CMS owes its success to a vast ecosystem of plugins and themes. These tools enable efficient creation of personalized websites for companies and individuals. Thousands of both free and paid plugins are available in the official WordPress repository and on platforms like ThemeForest [14]. The significant annual economic impact of these plugins and themes underscores their popularity [1]. Despite this, the security of these plugins has historically been overlooked.

```
1  <?php
2  Class PluginClass {
3    public function __construct(){
4      include_once DIR . "inc/common.php";
5      add_action('save_post',
6      array($this, "my_save_post"), 10, 2);
7    }
8    public function my_save_post($post_id, $post){
9      $limit = is_numeric($_GET['limit']) ? $_GET['limit'] : 1;
10     $order = $_GET['order'];
11     MyDb::getInstance()->getComment($post_id, $order, $limit);
12     $title = __($_GET['title']);
13     echo $title;
14   }
15 }
16
17 Class MyDb{
18   private static $instance;
19   public static function getInstance(){
20     if (!self::$instance)
21         self::$instance = new self();
22     return self::$instance;
23   }
24   public function getComment($id, $order, $limit){
25     global $wpdb;
26     $wpdb->query("SELECT comment FROM posts WHERE id=$id ORDER BY $order LIMIT
       ↪  $limit");
27   }
28 }
29
30 new PluginClass();
31 ...
32 ?>
```

Listing 2: A code example of WordPress plugin.

Listing 2 shows a WordPress plugin with Cross-Site Scripting and SQL injection vulnerabilities. During runtime, WordPress loads such plugins which then change its process using hooks. Two types of hooks are provided: Actions and Filters. Actions, like the `my_save_post` function in line 5 of Listing 2, trigger during specific WordPress core events. Filters enable data modification during WordPress execution. The plugins also utilize WordPress's helper functions for tasks like database queries (`$wpdb` in line 25 of Listing 2) and data sanitization. Notably, some string helper functions return arguments partially or entirely, affecting static code analysis accuracy. Without deep CMS knowledge, analysis tools may miss crucial vulnerabilities, as seen with phpSAFE [9] and RIPS [6] overlooking the ___ function's XSS vulnerability in line 12 of Listing 2.

PHP currently stands as the most popular scripting language for web applications [15], with many dynamic natures, including dynamic includes, arrays, and functions [6], are frequently utilized in WordPress plugins (e.g., line 4 in Listing 2). We will detail a string analysis approach to model these dynamic features in a later section.

Object-oriented programming (OOP) features have been integral to PHP since version 5.0, widely adopted in web applications including WordPress and its plugins up to the latest version 8.*. Listing 2 illustrates a typical OOP-style WordPress plugin. In this example, the plugin accesses the 'MyDb' class instance using the `MyDb::getInstance()` function and queries the database with the `getComment` method (line 11). Analyzing such PHP code necessitates handling OOP features like method calls and type inferences. Inaccurate handling of these features leads to challenges in precisely analyzing method calls like `getComment`. For instance, user input `$_GET['order']` flowing to the SQL query in `getComment` could be missed. Previous studies often treated OOP code as functional code, leading to extensive but inaccurate method searches and a high rate of false positives.

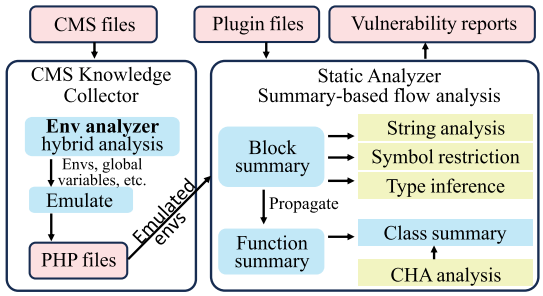

Fig. 1. Overview of PSSA.

III. System Design

In this section, we outline PSSA and detail each module.

A. System Overview

Fig. 1 depicts PSSA's architecture. PSSA is split into two main components: the CMS Knowledge Collector and the Static Analyzer. The CMS Knowledge Collector gleans essential CMS information like global objects and sink functions using static analysis and dynamic testing techniques. During the Static Analyzer phase, PSSA performs a context-sensitive, inter-procedural static analysis aimed at detecting taint-style vulnerabilities in plugins. This begins with a foundational summary-based interprocedural data flow analysis. In parallel, the system undertakes type inference and class hierarchy analyses to ascertain each variable's potential type. Such type data is subsequently disseminated through the data flow, aiding in the resolution of method call receivers. With this, PSSA conducts a taint analysis, integrating vulnerability sanitization/validation and context analysis, pinpointing genuine vulnerabilities.

B. CMS Knowledge Collector

The CMS Knowledge Collector is essential for web application plugin analysis in CMS, as it facilitates understanding and utilization of CMS APIs for security-critical operations, such as input sanitization and database interactions. Plugins, integral to CMS, leverage CMS-defined functions, including those for string manipulation and translation, which if not properly modeled can omit crucial data flows, leading to significant false positives. Previous studies, such as [9], have overlooked this aspect. By collecting sink functions, methods, global objects, and string functions during both static analysis and dynamic testing, the CMS Knowledge Collector enhances the analysis of plugins, ensuring a more comprehensive understanding of their operations.

The CMS framework we mentioned earlier poses inherent analytical challenges. For instance, plugin operations are injected into workflows via dynamic hooks. This makes it extremely hard to accurately simulate a plugin's contextual environment through pure static analysis. To tackle this, we built a WordPress plugin that replicates the same environment as other plugins. It dynamically collects global variables, argument functions, and other elements from the CMS context—elements that are hard to extract accurately with static analysis

Global object: CMSs typically offer APIs and global objects to facilitate plugin development. For example, WordPress provides numerous global objects available for general use. One such object is the global object `$wpdb`, which allows direct database querying without reestablishing a connection. During CMS runtime, global variables can be extracted by creating a plugin. To accomplish this, our plugin saves all the global objects and object classes with specific classes across multiple requests. However, primitive type global variables are not stored, as their static value is not pertinent to our subsequent analysis.

CMS sanitize and validation function: CMSs supply various helper functions to assist plugins in sanitizing or validating user-controlled data. Overlooking these functions could result in incorrect taint flow propagation. WordPress offers numerous sanitize functions, such as `sanitize_text_field`, and escape functions like `esc_html`. It also provides several widely used nonce verification functions, such as `wp_verify_nonce`, that help prevent XSS vulnerabilities. CMSs thoroughly document these functions [16], which frequently employ complex dynamic operations on strings. Therefore, we manually collect these functions.

Argument functions: RIPS [6] categorizes PHP built-in functions into several classes, among which argument functions may return their input parameters either partially or entirely (e.g., `trim()`). Accurately modeling these functions is essential for preserving taint propagation from arguments to return values, especially in CMS plugins that heavily rely on string manipulation, translation, and normalization APIs. Static analysis alone is often insufficient, as these functions typically operate on strings through iterative character- or token-level processing, which is difficult to model precisely. To address this limitation, we combine static analysis with dynamic testing: we first identify CMS functions whose return values may depend on their parameters and record their parameter positions and types. We then apply fuzzing inputs, including SQL injection and XSS payloads, to these functions at runtime and identify 131 argument functions in WordPress, such as `urldecode_deep`.

CMS sink function: In addition to directly calling the PHP primitive sink function, plugins may also invoke sink functions through CMS functions or methods. We conduct a static analysis of the CMS source code to identify all potential sink functions. Specifically, we compile all functions for which the function parameter or global variable might flow to the PHP primitive sink function through inter-procedural, context-sensitive analysis. We employ argument, sanitize, and validation functions to enhance the taint analysis. The static analysis method is identical to plugin analysis, which is addressed later. Previous research has manually modeled CMS sink functions [9]. Due to the absence of documentation on sink functions, manual investigation of the CMS source code is needed. This is a highly specialized and painstaking task which demands extensive professional knowledge and is time-consuming and error-prone. Our approach precisely and automatically addresses this issue.

C. Static Analyzer

We extend RIPS by enhancing the object-oriented analysis and implementing vulnerability-oriented, path-sensitive

analysis. This method offers improved accuracy for PHP programs and reduces false positives. We start by converting PHP source code into abstract syntax trees (ASTs) and then form control flow graphs (CFGs). Our subsequent static analyses are performed by navigating these CFGs.

1) Summary-Based Inter-Procedural Analysis: To perform a context-sensitive inter-procedural analysis and a vulnerability-oriented, path-sensitive analysis, we initially generate block summaries and function summaries for each basic block as well as for every user-defined function or method. These summaries consist of symbolic values corresponding to memory locations.

Block summary: A block summary records the effects of a basic block, such as the data flow and restrictions for symbols. We utilize it in subsequent static taint analysis and function summary construction. Our block summary extends previous work and consists of the following properties.

- *VariableCache:* Record the symbols corresponding to the variables visible in this block.
- *Constants:* Store all constants defined in this block.
- *IsExit:* Mark whether this block is an exit block.
- *ReturnValue:* Store the return symbol defined in this block we inferred before.
- *Restrictions:* Store all the symbols with their constraints in this block. We will set the restriction on the symbol extracted from the edge to the following block when simulating an edge.
- *PropWriteCache:* Record the assignment of the property of known and unknown object symbols in this block.

We perform a data flow analysis similar to RIPS to deduce the block summary. In comparison to previous work, we removed the DataFlow field and employ block summary propagation to transfer the data flow in VariableCache, reducing the instances of backtracking symbols. We introduce Restrictions summary to determine path-sensitive sanitization and validation, which will be discussed in Section III-C4. We utilize ArrayDimFetch and PropertyFetch symbols to manage array read and property read operations, respectively. Both symbols possess a base attribute that indicates the source of the read. The PropWriteCache in the block summary records data flow information between properties, keyed by the receiver's object symbol rather than by class, so that distinct instances of the same class are tracked separately—i.e., our property reasoning is object-sensitive in the standard sense [17].

Block summary propagation: Prior to summarizing a block, we transfer all `VariableCache`, `Restrictions`, and `PropWriteCache` from parent block summaries to the current block. Loop edges are disregarded since they have no significant impact on our static analysis. The propagation of `Restrictions` will be addressed later. For `VariableCache` and `PropWriteCache`, we merge the symbols of all parent blocks and pass all possible values to the current block.

Function summary: Function summaries store all inter-procedural information of a function, which are used for subsequent inter-procedural analysis. Given a function CFG, we first construct all the basic block summaries. Subsequently, we conduct a path-insensitive, intra-procedural data-flow analysis to infer function summaries. If a block has no following nodes, it must be the end of the function, return block, or exit block. Using these block summaries, we infer the function summary as follows.

- *ReturnValue:* A set of all possible return value symbols.
- *IsExitFunction:* The tag is set if a function exits in any path.
- *ChangedGlobals:* Record the symbol that this function may modify a global variable of a certain name into.
- *SensitiveParams:* Record the parameters that flow to the sink call in this function and the vulnerability type.
- *SensitiveGlobals:* Record the global variables that flow to the sink call in this function and the vulnerability type.
- *SensitiveProperties:* Record the PropertyFetch symbol that flows to the sink call in this function, where the Object base of the symbol is the function parameter.

Inter-procedural analysis: Following the successful construction of the function summary, we can simulate the effects of a function call in a context-sensitive manner. Upon encountering a function call, we replace the portion associated with the function parameters with arguments in the function summary's return value, thereby obtaining the function call's return value. Using the IsExitFunction tag, we determine whether to mark the block as `IsExitBlock`. Subsequently, the function parameter is replaced with the modified global variable symbol, which is then added to the global variable symbol list. For the `PropWriteCache` of the function parameter symbol in the function's end block, we apply the property write data to the parameter. In the case of a method call, the property of the object is applied to the receiver object symbol. This process achieves an implementation of inter-procedural data flow analysis while effectively preserving the crucial details of key strings.

Taint analysis: While performing intra-procedural data-flow analysis, we also conduct taint analysis, tracking function parameters and global variables to the sink function. And then add sensitive parameters and sensitive global variables to the function summary. Consequently, this function can also be used as a sink function the next time it is called.

2) Object-Oriented Analysis: Type inference: PHP is a language with weak typing, the values and types of variables are determined at runtime. So it is challenging to figure out all the types precisely. But after version 5.0, PHP allows type definition of function arguments and provides type definition of function return values after version 7.0. We find that more and more modern PHP programs use type definitions to standardize their code. In this case, we conduct a lightweight, over-approximate type inference for PHP.

Initially, we extract all classes from the source code and perform a class hierarchy analysis. Next, we extract function parameters and return values with type definitions. These types will remain unchanged during subsequent type inference analysis. As PHP variable types can change dynamically at runtime, we maintain a type list for each variable symbol. When an object is created in the code, its type is stored in its symbol. We record all possible types upon merging blocks, even though this is highly approximate. During inter-procedural analysis, the type information is passed to the calling symbol along with the return value symbol. Additionally, we collect restrictions on

variables for the instance of conditional statement in the symbols of corresponding blocks.

Receiver analysis: Constructing a precise call graph for object-oriented PHP source code is challenging. During interprocedural analysis, when encountering a method call, we must identify the corresponding class method according to its receiving symbol. For symbols with inferable type lists, we find the corresponding methods for all classes in the type list and apply all class methods with relevant method names. For symbols without a discernible type, such as function parameters without type definitions, class attributes, and global variables, we search for all possible methods sharing the same method name. We then verify the number of parameters, parameter types, and other restrictions before applying the final class method list. Each method's return value from a call site is merged into the return symbol. We believe that such an over-approximation strategy will not introduce a considerable false-positive rate for vulnerability analysis, as can be observed in later evaluations.

Regarding static method calls, we extract the class name and method name and locate the corresponding class for execution according to the class hierarchy analysis results. Dynamic method calls are also prevalent in PHP, and the following section will detail how we address them.

3) Dynamic Features and String Analysis: In dynamic scripting languages such as PHP, various dynamic features exist, including dynamic includes, function calls, and code, complicating static analysis [6], [9], [11], [18], [19]. We conduct string analysis concurrently with intra-procedural data flow analysis, utilizing `ValueConcat` Symbols to document string concatenation operations. Efforts are made to preserve string information when encountering built-in string manipulation functions.

A comprehensive simulation of both built-in PHP and CMS string manipulation functions enables more accurate string analysis. When faced with dynamic includes, function, or method calls, we infer the file names and functions to be called based on string analysis results. For aspects not discernable in string analysis, such as function parameters, we apply regular matching to identify suitable file and function names. Similar to RIPS, newly included files are treated as function summaries, and variables within these files are integrated into the include's execution context. For other dynamic features, such as variable variables and dynamic code generation, we attempt to construct and analyze strings as ordinary code.

4) Vulnerability-Oriented Path-Sensitive Analysis: We introduced earlier that it is challenging to handle path-sensitive sanitization and validation in the current static analysis. This section will present our novel vulnerability-oriented path-sensitive analysis to recognize those path-sensitive security mechanisms.

Simulate block edges: PHP has a large number of built-in input verification functions, which can verify the input type, string matching, string length, blocklist or allowlist verification, etc. The return values of these functions are all true or false. We simulate the return values of these functions in Boolean symbols and store the constraints of their arguments in the Boolean symbols. For example, for the `is_numeric($var)` statement, we first resolve the symbol represented by the `$var` variable and then store the *Type* constraint of the `$var` variable symbol in the Boolean symbol. This symbol can keep constraints information after propagating the data flow. For example, the symbol is passed to other conditional statements in a variable or passed through return.

When a condition block depends on a Boolean symbol, we pass the constraints in the Boolean symbol to the block summary of the if block and pass the negative constraints to the block summary of the else block. It should be noted that we consider that each conditional statement has a corresponding if block and an else block, although the else block may be empty. The restrictions field in the block summary stores the constraints of certain symbols in the block. If there is a constraint on a certain tainted symbol in the block, then the symbol will be marked as sanitized in that block. We only analyze the restrictions of SourceNode (source input, function parameters, global variables) symbols, because the analysis of other symbols is wasteful and unnecessary.

Restrictions propagation: We analyze path-sensitive security mechanisms through Restrictions propagation. Here we only consider one type of restriction $\mathrm{C_i}$, so there will be many restriction values for this restriction $\mathrm{C_{i1}}, \mathrm{C_{i2}} \cdots \mathrm{C_{ij}} \in \mathrm{C_i}$. When we analyze each basic block, we will merge all the restrictions of its parent block, formulated as: $\mathrm{Block}_c = \mathrm{PBlock1}_c \cup \mathrm{PBlock2}_c \cdots \cup \mathrm{PBlockn}_c$ This means that we must analyze all parent blocks of a block before we can analyze this block. For loops, we ignore the extra loop edge. Suppose there are both positive and negative restrictions with the same restriction type for a certain symbol when merging restrictions. In that case, We will remove this restriction on that symbol in this block $\mathrm{C_{ij}} \cup ! \mathrm{C_{ij}} = \emptyset$. If a symbol is sanitized in a block, and there are restrictions on it, we will convert these sanitization tags into restrictions and propagate them to the next block. In this way, we can also propagate the path-sensitive sanitization information to the following block.

Now we can solve the Path-sensitive Termination situation and Path-sensitive Sanitization situation. Suppose code exits directly in a basic block, such as directly return or exit in Listing 1 lines 12-13 or unset a variable in line 5. In that case, this basic block will not transfer the restrictions of this symbol to the following block. For path-sensitive sanitization methods like Listing 1 lines 3-7, the restrictions of subsequent blocks can be formulated as:

$$\begin{aligned} \mathrm{Block}_c &= \mathrm{C}_{numeric} \cup (! \, \mathrm{C}_{numeric} \cap \mathrm{C}_{sla\_quo}) \\ &= \mathrm{C}_{numeric} \cup \mathrm{C}_{sla\_quo} \end{aligned} \tag{1}$$

We can conclude that the next block's restriction on this `$var` variable symbol is either to limit its type to numeric or to have a sanitization tag. For the last case of Path-sensitive Validation, we record the assignment of Boolean symbols and integers of 0 or 1 in each block. The corresponding Boolean symbol will record the restriction information of this block. When this symbol is used as a branch condition, these restrictions are passed on to subsequent blocks.

Multiple restrictions types: We can divide the commonly used restrictions in PHP into the following categories.

```
1  <?php
2  function funcl($var){
3    if(!is_numeric($var) && !in_array($var, ["red","blue"])){
4      if(is_string($var) && strlen($var) < 3){
5        $var = addslashes($var);
6      }
7    }
8    sink($var);
9  }
```

Listing 3: A code snippet for multiple restrictions types.

- *Type restrictions:* Built-in functions, e.g., `is_numeric` and `is_bool`, check the type of input parameters, and `instanceof` checks the type of the object.
- *Sanitization restrictions:* In order to detect Path-sensitive Sanitization, we convert the sanitization tag into sanitization restriction and spread it in the block summary.
- *Static value restrictions:* There are restriction methods to restrict the input to a static value, such as the common use of `in_array` and other functions to check whether the input is in a whitelist and the use of the string comparison function strcmp or directly use the $==$ operator to compare.
- *Length restrictions:* In PHP, we can use the strlen built-in function to limit the length of the input. If the input length is limited to short enough, it is not available for attackers.
- *String match restrictions:* String search and regular matching can be used to validate if it contains malicious content.

We only discussed the path-sensitive security measures of a single restriction type above. Although most of the security check for a variable is a single restriction type, there are also combinations of restrictions. PSSA handles multi-restriction types in the same way. First, it collects the restriction information into the block summary, e.g., the restriction information in the block summary on line 8 is $\mathrm{C}_{numeric} \cup \mathrm{C}_{staticvalue} \cup! \mathrm{C}_{string} \cup! \mathrm{C}_{length} \cup \mathrm{C}_{sla\_quo}$. Then, when a controllable symbol encounters a sink call, it checks the restrictions in this block and the sanitization tag of the symbol. If the analysis has conflicts or there are conditions that cannot be analyzed, we assume that it may be unsafe.

Limitations: Our vulnerability-oriented path-sensitive analysis only analyzes conditional constraints related to user input or function parameters, so it does not significantly affect the performance of static analysis. In our subsequent analysis, it has demonstrated that most path-sensitive security measures are related to user input or function parameters. Therefore, we cannot avoid false positives caused by other constraints. In addition, in order to improve the code coverage, we will analyze all the code of the plugin. However, a file could include another file, and then the order to analyze file would influence the variable restrictions. Therefore, we will not handle the restriction propagation across files, which in theory will also lead to some false positives. Moreover, since we cannot solve complex constraints, we are also unable to handle complex conditions.

5) Vulnerability Context Analysis: We perform vulnerability context analysis based on the results of the above string analysis. Many vulnerabilities require contextual analysis of the strings flowing into the sink call to distinguish the types of vulnerabilities in detail or determine whether they are vulnerable or not. For example, we need to determine whether the controllable source appears between quotation marks and record the quotation mark type for SQL injection. For XSS, we need to analyze whether the controllable source appears in the tag attribute or the javascript code, and we need to determine whether it is wrapped in single and double quotation marks. Different types of vulnerabilities will use different sanitization methods to compare with the sanitization tag we recorded accurately.

D. Implementation

PSSA was developed using more than 20,000 lines of PHP code because the PHP language has a more complete ecosystem for parsing AST and building CFG, which saves us a lot of initial development. We employed PHP-Parser [20] to parse PHP source code into ASTs and php-cfg [21] to create the CFGs, both of which are recognized PHP static analysis tools [22]. The Object-Oriented taint and vulnerability analyses are conducted by navigating the CFGs. For CMS knowledge collection, we apply the same static analyzer and have crafted a WordPress plugin for dynamic testing. We will make all source codes available on GitHub for future research after being accepted.

IV. Performance Evaluation

A. Experimental Setup

Dataset: For ground truth evaluation, we collected 24 publicly-reported vulnerable WordPress plugins. As the similar plugin vulnerability static analysis tool, phpSAFE, does not disclose its test set, we selected an appropriate test set ourselves. We followed three principles in selecting vulnerable plugins: (1) it should be reported at least one SQL injection or Cross-Site Scripting vulnerability in the last five years, and we could download the corresponding vulnerable version and successfully reproduce the vulnerability; (2) it should have at least one million downloads or one hundred thousand active installs recorded on the official WordPress website; (3) it should be still available and under maintenance. These requirements ensure that the plugins in the dataset have large and complex codebases, which can more effectively verify the effectiveness of our tools.

We compare PSSA with the latest open-source versions of RIPS [6], phpSAFE [9] and TChecker [7], which are three advanced static analysis tools for vulnerability detection in PHP applications or CMS Plugins. It's important to highlight that phpSAFE lacks the capability to analyze plugins that dynamically include PHP files, leading to lower code coverage when only the main plugin file is analyzed. However, the phpSAFE documentation suggests that analyzing all PHP files could enhance code coverage. Therefore, we modified its source code from GitHub to enable comprehensive file analysis. Additionally, since RIPS and TChecker does not inherently model WordPress's database query API, we manually integrated WordPress's database query functions into their sink configuration file to augment its analysis capabilities. In addition, the open-source version of TChecker contains several implementation bugs (as also documented in other related studies [23]), such as flaws in inter-procedural taint propagation. To ensure a fair comparison, we have rectified these

TABLE I
GROUNDTRUTH RESULTS OF PSSA. THE COLUMN OF "RES" SPECIFY WHETHER THE TOOL CAN DETECT THE VULNERABILITY; √ INDICATES DETECTION, WHILE ✗ SIGNIFIES NO DETECTION. THE "RES" COLUMN CONTAINS TWO VALUES IN PARENTHESES. THE FIRST VALUE IS THE NUMBER OF FALSE POSITIVES FOR SQL INJECTIONS, AND THE SECOND VALUE IS THE NUMBER OF FALSE POSITIVES FOR CROSS-SITE SCRIPTING VULNERABILITIES.. THE COLUMN OF "TIME" AND "MEM" REPRESENT THE TIME AND MEMORY CONSUMED BY THE ANALYSIS RESPECTIVELY.

Plugin Name(Version)	Downs	Vuln	Fils	Locs	PSSA			RIPS			phpSAFE			TChecker		
					res	time	mem	res	time	mem	res	time	mem	res	time	mem
MailChimp for wp(4.0.10)	30+	XSS	567	66484	✔(0,0)	4.9s	70M	✗(0,0)	0.6s	3M	✗(1,0)	1.6s	5M	✔(0,0)	2.1s	76M
MailChimp for wp(4.1.6)	30+	XSS	597	68099	✔(0,0)	9.5s	70M	✗(0,0)	0.6s	3M	✗(1,0)	1.6s	5M	✗(0,0)	2.3s	87M
UpdraftPlus(1.16.65)	70+	XSS	1994	654976	✔(1,0)	53.3s	321M	✗(16,0)	16.5s	35M	✗(13,0)	18.1s	36M	✗(7,2)	33.4s	338M
Elementor(2.8.4)	180+	XSS	1781	392316	✗(0,0)	43.3s	196M	✗(1,0)	2.3s	7M	✗(1,0)	1.6s	6M	✗(1,1)	5.1s	63M
LiteSpeed Cache(4.4.3)	20+	XSS	557	102773	✔(0,0)	39.5s	214M	✗(3,10)	2s	9M	✗(3,2)	2.2s	8M	✗(3,1)	13.7s	137M
Limit Login Attem...(2.15.1)	20+	XSS	13	4050	✔(0,0)	39.5s	214M	✗(0,0)	0.4s	7M	✗(0,0)	0.4s	1.6M	✗(0,4)	4.8s	42M
W3 Total Cache(2.1.3)	30+	XSS	6342	1228110	✔(0,0)	248s	514M	✗(20,0)	127.2s	282M	✗(0,0)	168.2s	121M	✗(0,1)	43.4s	343M
W3 Total Cache(2.1.4)	30+	XSS	12684	2456318	✔(0,0)	357s	854M	✔(5,0)	55.0s	87M	✗(0,0)	74.6s	59M	✔(0,1)	45.1s	377M
iThemes Security(7.0.2)	20+	SQLi	3128	246584	✔(0,0)	52.1s	224M	✔*(10,1)	4.8s	9M	✗(2,1)	6.0s	8M	✔*(0,0)	12.4s	279M
Popup Maker(1.6.4)	10+	XSS	6622	1421652	✔(0,0)	55.4s	240M	✔(10,1)	7.1s	35M	✔(14,0)	9.0s	36M	✔(1,1)	12.3s	148M
Broken Link Checker(1.11.8)	10+	XSS	178	61910	✔(0,0)	15.2s	136M	✗(4,0)	3.1s	23M	✗(1,2)	3.2s	13M	✗(0,0)	21.1s	152M
WP Statistics(13.07)	10+	SQLi	2478	271530	✔(0,0)	113.5s	688M	✔*(0,8)	6.5s	101M	✗(3,1)	4.7s	89M	✗(0,4)	7.8s	116M
Simple Custom CSS...(3.2)	4+	XSS	18	7070	✔(0,0)	3.3s	40M	✔(2,0)	0.2s	4M	✔(0,0)	0.7s	7M	✔(0,0)	3.0s	42M
Variation Swatches...(1.0.61)	3+	XSS	52	13630	✔(0,0)	5.5s	50M	✗(0,0)	0.4s	6M	✗(0,1)	0.5s	6M	✗(0,0)	2.5s	60M
Header Footer Code...(1.1.13)	2+	SQLi	8	2960	✔(0,0)	2.8s	50M	✗(0,0)	0.1	4M	✗(0,0)	0.1	3M	✗(0,0)	4.4s	48M
GTranslate(2.8.51)	7+	XSS	16	6702	✔(0,0)	4.8s	58M	✗(6,0)	0.3s	7M	✗(5,0)	3.1s	6M	✗(4,1)	5.1s	68M
LoginPress(1.1.13)	2+	SQLi	154	38494	✔(0,0)	11.4s	86M	✔*(0,8)	0.5s	6M	✗(9,0)	0.2s	4M	✔*(0,2)	6.5s	90M
Ultimate Member(2.1.19)	6+	XSS	5660	930492	✔(2,0)	73.8s	628M	✗(55,2)	4s	14M	✗(9,0)	5.2s	9M	✗(2,0)	15.3s	340M
Responsive Lightbox...(1.7.1)	4+	XSS	16	13318	✔(0,0)	6.5s	60M	✔(1,0)	0.6s	10M	✔(1,0)	0.5s	4M	✔(0,0)	6.7s	140M
Popup Builder (3.73)	7+	XSS	734	226196	✔(0,0)	19.9s	170M	✔(2,13)	3.6s	11M	✗(6,0)	4.2s	12M	✗(0,2)	9.4s	136M
Popup Builder (3.44)	7+	SQLi	678	208846	✔(0,0)	22.4s	168M	✔*(2,13)	3.5s	12M	✗(6,0)	2.8s	9M	✔*(1,0)	9.9s	149M
Pagelayer(1.3.4)	3+	XSS	150	131236	✔(0,0)	17.1s	152M	✔(9,0)	2.8s	28M	✔(13,1)	9.7s	32M	✔(5,3)	21.4s	170M
ShareThis Dashboard...(2.5.1)	5+	XSS	238	27782	✔(1,0)	5.6s	54M	✔(0,0)	0.3s	4M	✔(5,0)	0.1s	3M	✔(0,0)	4.1s	70M
Favicon by RealFav...(1.3.21)	2+	XSS	132	25332	✔(0,0)	3.0s	42M	✔(0,0)	0.5s	5M	✔(0,0)	0.4s	6M	✔(0,2)	4.0s	53M

issues to the fullest extent possible by strictly adhering to the solution proposed in the original paper.

For new vulnerability detection evaluation, we selected all plugins from the popular category in the official WordPress plugin repository [24]. It included widely-used plugins in WordPress, encompassing functionalities like e-commerce, form generation, website analytics, security, and more. These plugins, often exhibiting complex object-oriented code structures, aptly represent contemporary plugin development. Considering some plugins span over 2 million lines of code, manual security audits become immensely challenging. We deploy PSSA and perform our experiments on an Intel Core i7-10750H CPU 2.60 GHz, 32 GB of memory, running Windows OS with PHP 7.4 installed.

B. Groundtruth Evaluation

We benchmarked PSSA against phpSAFE [9], RIPS [6] and TChecker [7], which are leading static analysis tools for detecting vulnerabilities in PHP. We obtained their source codes from GitHub or official website and utilized them as optimally as described in their respective documentation. Notably, tools developed in PHP were executed in an identical environment to that of our tool for a fair comparison.

Evaluation Results: The outcomes of our vulnerability analysis across a ground truth dataset, involving PSSA, RIPS, phpSAFE and TChecker, are captured in Table I. This table organizes data as follows: the first column lists the plugin names and their versions, the second column represents the number of times the plugin was downloaded when the vulnerability was discovered [25]. The unit of this column is millions. The third column identifies the type of vulnerabilities present, and the fourth and fifth columns detail the requisite number of files and lines of code for analysis.

PSSA showcased notable efficiency and precision in identifying SQL injection and XSS vulnerabilities within the dataset, successfully detecting 23 out of 24 vulnerable instances with a minimal rate of false positives. A noteworthy mention is the analysis of the `Elementor 2.8.4` plugin, where PSSA's false negatives were observed—none of RIPS, phpSAFE and TChecker managed to detect this particular vulnerability. We plan to delve deeper into this case in subsequent sections. Among the 24 identified plugin vulnerabilities, RIPS flagged 12, phpSAFE identified 6, and TChecker detected 13, with all three tools exhibiting higher instances of false positives for reasons we will explore in further discussion.

Performance: The three tools all perform pure static analysis, thus there is no significant difference in terms of time consumption. However, PSSA stands out due to its more complex and thorough path-sensitive analysis, resulting in higher time consumption and memory usage compared to the other three tools. Nevertheless, these additional time and memory costs are acceptable.

False negatives: None of the four tools detected the vulnerability in `Elementor 2.8.4`. This issue occurs because Elementor generates files through dynamic includes and passes the tainted variable to the dynamic file. The vulnerable code appears in Listing 4. Lines 3 generate the template name using the `$template` parameter and then include the dynamic file `system-info/templates/html.php`. The templates file outputs the value of the `$reports` array, which originates from the user-controllable source in line 7. Accurately analyzing a dynamic scripting language like PHP is challenging for static analysis tools. The `$template_path` value can only be determined accurately at runtime. Since PSSA analyzes each function once to generate a function summary, it cannot obtain the value of `$template` when

```
1  <?php
2  public function print_report( $reports, $template) {
3    $template_path = $this->get_settings( 'dirs.templates' ). $template . '.php';
4    require $template_path;
5  }
6
7  $this->print_report( $source, 'html' );
8  ?>
9
10 `system-info/templates/html.php`
11 <?php
12 echo $reports[$report_name]['label'];
13 ...
14 ?>
```

Listing 4: Simplified code snippet of Elementor 2.8.4: systeminfo/main.php file.

```
1  <?php
2  class UM{
3    static public function instance() {
4      if ( is_null( self::$instance ) ) {
5        self::$instance = new self();
6        self::$instance->_um_construct();
7      }
8
9      return self::$instance;
10   }
11   function fields() {
12     if ( empty( $this->classes['fields'] ) ) {
13       $this->classes['fields'] = new um\core\Fields();
14     }
15
16     return $this->classes['fields'];
17   }
18 }
19 function UM() {
20   return UM::instance();
21 }
22 echo UM()->fields()->display_view( 'profile', $args );
23 ... ?>
```

Listing 5: Simplified code snippet of Ultimate Member 2.1.20.

generating the summary. So it cannot accurately include the correct file.

Comparison with SOTA tools: RIPS does not support the analysis of object-oriented code, treating method and function calls similarly, which leads to a significant number of incorrect function analyses and false negatives. phpSAFE's support for object-oriented features is also limited, and it does not support dynamic include syntax, making it challenging to analyze complex PHP code. Plugins `MailChimp for wp 4.0.10`, `Limit Login Attempts Reloaded 2.15.1`, and `Ultimate Member 2.1.20` are false negatives due to the insufficient object-oriented support that neither RIPS nor phpSAFE can analyze correctly. We take `Ultimate Member 2.1.20` as an example. Listing 5 presents the simplified plugin vulnerability code.

Line 22 in Listing 5 directly outputs the return value of `display_view` method. Thus, an attacker can control the return value, leading to a Cross-Site Scripting vulnerability. The vulnerable code utilizes an object-oriented chained call style. PSSA can determine the return type of `UM()->fields()` in the fields function summary as the `UM` class. Simultaneously, after performing type inference analysis, we deduce that the `classes['fields']` attribute type of the UM class may be `um/core/Fields`, as indicated by the assignment in line 13 of Listing 5. Thus, the call to the `display_view` method can be successfully resolved, and the executed method is `um/core/Fields::display_view`. RIPS and phpSAFE do not support or provide incomplete support for object-oriented code. Consequently, they could not resolve the `display_view` method, resulting in false positives.

As for RIPS and TChecker, they lack CMS knowledge such as the built-in sink functions of CMS (e.g., `$wpdb->query`), thus failing to detect certain vulnerabilities. We regard this as a trivial issue: since these sinks are well-documented in WordPress official documentation, we manually supplemented them into the aforementioned tools and marked the corresponding entries with an asterisk (*) in Table I). However, For other CMS function like `add_query_arg`, attacker can control its return value. Though not documented in the WordPress official document, PSSA can discover this source through the CMS Knowledge Collector stage, but other tools cannot get this source easily. By default, RIPS treats unknown functions that tainted data passes through as argument functions. Even though RIPS does not recognize the WordPress argument function, it can still accurately connect the data flow. However, this approach leads RIPS to report numerous false positives.

Regarding phpSAFE, we found that it does not support the syntax of dynamic include files in PHP and often struggles with cross-file function and method calls, such as in `W3 Total Cache 2.1.3`. Additionally, phpSAFE frequently loses taint analysis data for multiple string concatenation syntax. Since web developers typically concatenate query statements in PHP code, phpSAFE cannot detect many SQL injection vulnerabilities. Simultaneously, phpSAFE cannot properly handle argument functions. In Popup Builder 3.73 and Limit Login Attempts Reloaded 2.15.1, phpSAFE loses data flow because it cannot handle the `esc_sql` and `sanitize_text_field` WordPress argument functions, respectively.

For TChecker, in addition to the false negatives caused by its lack of CMS context knowledge, the missed detection of two vulnerabilities in plugins such as `Header Footer Code` can be attributed to its partial inter-procedural taint analysis. In pursuit of analysis efficiency, this tool only tracks taint-related functions during function call analysis (see TChecker [7], §4.3.2, "Preprocessing Target Functions"): a callee is skipped when its body has no tainted parameter, no source access, and writes only to object properties. A common consequence in plugin code is that a class constructor which sets an object field to a constant string—a sink name, a table prefix, a callback identifier—has its write silently dropped at later read sites of the same field. When that field is then used as the target of a dynamic call, TChecker cannot resolve the call target and, following its own policy [7, §4.2.1], ignores the call site, missing the sink. The SQLi sink in `Header Footer Code 1.1.13` is missed by TChecker as one instance of this constructor-set-constant-then-dynamic-call pattern, while PSSA reports it. In PSSA, the constructor is summarised in the same way as any other function: its `PropWriteCache` records the property write and is applied to the new object symbol at the `new` site, so the constant remains visible at downstream reads and the dynamic call's target is resolved via string analysis.

Meanwhile, TChecker also has flaws in its taint propagation rules. Specifically, it fails to properly handle the for-loop traversal of tainted arrays, resulting in two false negatives. To be fair, there are four additional vulnerabilities that TChecker should theoretically be able to detect, yet it failed to identify them due to flaws in its open-source implementation.

TABLE II
FALSE POSITIVES ABOUT THREE TOOLS

	FP_p	FP_k	FP_f	FP_c	FP_o
PSSA	2	0	0	2	0
RIPS	15	92	63	15	5
phpSAFE	5	9	49	5	26
TChecker	18	10	13	10	0

False Positives: As shown in Table I, it is evident that PSSA can detect more vulnerabilities while producing fewer false positives. This is because PSSA can analyze more complex modern PHP code and filter out a greater number of false positives. We have categorized the causes of these false positives into five groups, listed in Table II. FP_p represents false positives resulting from unrecognized path-sensitive sanitization or validation. FP_k signifies false positives caused by insufficient CMS knowledge, such as generic CMS sanitization functions. FP_f indicates incorrect false positives arising from flawed taint flow. RIPS and phpSAFE treat unknown functions that tainted data passes through as argument functions, which can lead to numerous false positives. FP_c, denotes false positives caused by unrecognized vulnerability context. TChecker generates false positives in this scenario due to inaccurate string analysis caused by its partially inter-procedural analysis. The last factor, FP_o represents other factors potentially causing false positives, such as incorrectly configured sources.

Table II shows that RIPS, lacking CMS knowledge configuration, has many false positives in FP_{know}. For example, RIPS misses the WordPress sanitization function `esc_html`, leading to taint flow errors. Another common false positive in FP_{know} is the nonce check for Cross-Site Scripting. RIPS and phpSAFE also often misinterpret unknown functions as argument functions, resulting in taint flow inaccuracies. All three tools likewise generated a considerable number of false positives in this regard, attributable to their path-insensitive analysis. While PSSA outperforms both in several areas, it's not immune to false positives due to the challenges of static code analysis in dynamic languages, such as the issue seen with `GTranslate 2.8.51` related to a dynamic value in path sensitivity.

C. Unknown Vulnerability Detection

In the ground truth evaluation experiments, PSSA's effectiveness is well established. Consequently, we employ PSSA for vulnerability detection within a more extensive plugin dataset. We crawled all plugins within the popular category from the official WordPress free plugin repository, amassing 980 plugins in total. As WordPress has validated these plugins for widespread use, their vulnerability impact is notably extensive. Subsequently, we applied the PSSA system to these plugins to detect SQL injection and Cross-Site Scripting vulnerabilities, identifying 178 vulnerabilities across 79 plugins, including 30 SQL injection and 148 Cross-Site Scripting vulnerabilities. Each detected vulnerability was manually verified for validity before reporting all newly discovered vulnerabilities to WPscan [26], the CVE assignee. WPscan has contacted all vendors to remediate these issues and assigned 82 CVE IDs to the identified vulnerabilities. It is important to note that when reporting multiple vulnerabilities of the same type within a single plugin at once, we receive one CVE ID.

```
1  <?php
2  function Ninja_Forms(){
3    return Ninja_Forms::instance();
4  }
5  class Ninja_Forms{
6    public static function instance(){
7      return new Ninja_Forms;
8    }
9    public function form($id=''){
10     return new ModelFactory($wpdb,$id);
11   }
12   public function sub($id=''){
13     $this->_object = new Submission($id,$form);
14     return $this;
15   }
16   public function get(){
17     return $this->_object;
18   }
19 }
20
21 $sub = Ninja_Forms()->form()->sub($nf_sub)->get();
22 if (isset($_POST['fields'])) {
23   $post_fields = WPN_Helper::esc_html($_POST['fields']);
24   foreach($post_fields as $field_id=>$user_value){
25     $sub->update_field_value($field_id, $user_value);
26   }
27 }
28 ... ?>
```

Listing 6: Simplified code snippet of Ninja Forms 3.6.3.

As illustrated in Table III, PSSA identifies numerous in-the-wild vulnerabilities with minimal false positives. PSSA effectively manages most false positives resulting from path-insensitive analysis and experiences no false positives due to CMS knowledge. In the subsequent paragraphs, we present two representative vulnerability cases and examine the discovered false positives in detail.

Experimental results demonstrate that our tool effectively analyzes complex modern plugin code, such as Ninja Form [27], which has garnered 30 million downloads. PSSA investigated this plugin and discovered an SQL injection vulnerability. Listing 6 displays a simplified version of the vulnerable code in `Ninja Form3.6.3`. Analogous to the Ultimate Member analysis, PSSA determines the `$sub` variable type on line 21 through the content of Listing lines 2 to 19 and successfully resolves the method call on line 25. The first parameter of the `update_field_value` method call originates from the `post_field` variable, derived from the user-controllable value `$_POST['fields']` key value. PSSA successfully models the source value's iterative operation, and the first parameter of the `update_field_value` method is concatenated into the database query statement without proper sanitization after multiple function calls, resulting in an SQL injection vulnerability. This case's challenge lies in inferring the `$sub` variable type via type inference, necessitating the analysis of all functions' return value types in lines 2 through 19 of Listing 6 during data flow analysis, a task that poses a significant challenge for other static analysis tools.

`ProfilePress 3.2.2` vulnerability exemplifies a Cross-Site Scripting issue in a widely-used plugin with over 300,000 installations [28]. A concise look at the vulnerability, as shown in Listing 7, reveals that the `get_forms_by_builder_type`

TABLE III
NEW VULNERABILITY DETECTION

	Plugins	TP	CVE	Popular	False Positives							FP Rate
					Knowledge	Path	Flow	Context	oop	Dead code	All	
SQLi	22	30	24	6	0	4	0	3	0	5	12	0.28
XSS	63	148	56	24	0	18	7	27	6	16	74	0.33

```
1  <?php
2  class AjaxHandler
3  {
4    public function get_forms_by_builder_type(){
5      $builder = sanitize_text_field($_POST['data']);
6      $this->menu_bar($builder);
7    }
8
9    public function menu_bar($builder){
10     $builder = sanitize_text_field($builder);
11     echo "<a href='#' data='$builder'>msg</a>";
12   }
13 }
14 ... ?>
```

Listing 7: Simplified code snippet of ProfilePress 3.2.2.

function is accessible by any user. On line 5, WordPress provides the `sanitize_text_field` function, which cleans up some user inputs, such as <. However, it fails to escape both single and double quotes. The unchecked `builder_type` variable gets transferred to the `menu_bar` function and subsequently reflects in the attributes of an HTML `a` element. By exploiting this, attackers can inject attributes into the `a` element using payloads like `' id=x tabindex=1 onfocus=alert(1) a='`, which can execute malicious JavaScript on the client-side. Notably, without the knowledge of the specific behavior of `sanitize_text_field` obtained during the CMS Knowledge Collector stage, most static analysis tools would miss this vulnerability.

False positives: Table III details the false positives in detecting new vulnerabilities. Results suggest a false positive rate of 28% for SQL injection vulnerabilities and 33% for Cross-Site Scripting vulnerabilities. Despite this, the detection accuracy remains commendable. A significant portion of false positives stems from vulnerability context, especially with XSS vulnerabilities. For SQL injection, we can typically discern between character and numeric injections using string analysis. Only a few instances exist where we can't accurately infer the context due to dynamic cross-function string generations. Determining the output context becomes challenging for Cross-Site Scripting vulnerabilities since many plugins output content across functions. Additionally, there's a notable amount of obsolete, untriggerable vulnerabilities in the code.

D. Path-Sensitive Evaluation

PSSA effectively identifies common path-sensitive security measures and significantly reduces false positives in vulnerability detection. Using popular WordPress plugins as the dataset, we compared vulnerability-oriented path-sensitive analysis with path-insensitive analysis (Table IV). Results show that false positive rates for XSS and SQL injection decrease notably, with SQL injection false positives dropping from 65% to 28% , eliminating 42 and 36 false alarms respectively, without introducing

TABLE IV
PATH-SENSITIVE ANALYSIS EVALUATION

Action	XSS			SQLi		
	Vuln	FP	Rate	Vuln	FP	Rate
Apply	222	74	0.33	42	12	0.28
Not	253	116	0.46	74	48	0.65

TABLE V
PATH-SENSITIVE SECURITY MEASURES

	Sanitize	Terminate	Validate	Inter
XSS	18	13	7	4
SQLi	31	2	0	3

```
1  <?php
2  $limit = is_numeric( $_POST['limit'] ) ? $_POST['limit'] : 200
3
4  $order = $_GET['order'];
5  $expected_order_values = array( 'asc', 'desc' );
6  if (!in_array(\$order, $expected_order_values)){
7    $order = 'desc';
8  }
9  ... ?>
```

Listing 8: Path sensitive sanitization.

```
1  <?php
2  function sanitize_value($val) {
3    $valids = ["DESC", "ASC"];
4    if (in_array( $val, $valids)) {
5      return $val;
6    }
7    return $valids[0];
8  }
9  $sort_order = sanitize_value($_GET['order']);
10 ... ?>
```

Listing 9: Example of interprocedural path sensitive.

any false negatives. Further analysis (Table V) reveals that path-sensitive sanitization is the most frequently used defense, particularly against SQL injection, while developers also adopt termination and validation measures to prevent XSS. Complex interprocedural sanitization measures appear less common, but overall, path-sensitive techniques play a central role in improving accuracy and reducing manual auditing efforts.

Sanitization and interprocedural measures: Path-sensitive sanitization is commonly applied to restrict user-controlled data, either by enforcing numeric values with default fallbacks or by validating specific inputs using functions such as `in_array` or `array_key_exists`, as shown in Listing 8. These practices are widely used to prevent injection vulnerabilities, and PSSA accurately identifies them to minimize false positives. In addition, developers often employ interprocedural strategies, such as defining path-sensitive functions (e.g., `sanitize_value`) that validate input against predefined arrays and restrict variables like `sort_order` to fixed values (Listing 9). Together, these techniques illustrate how both local and interprocedural sanitization enhance security by constraining user input across different execution paths.

```
<?php
if (!wp_verify_nonce($_GET['nonce'], 'nonce') ) {
  wp_die('Security Token Error!');
}

$nonce_verified = wp_verify_nonce($nonce, 'review');
...
if ($nonce_verified){
  ...
}
... ?>
```

Listing 10: Example of termination and validation.

Termination and validation measures for security: Path-sensitive termination and validation play a key role in preventing Cross-Site Scripting (XSS) vulnerabilities. In WordPress, the `wp_verify_nonce` function, originally intended to mitigate CSRF attacks, is widely used as an additional safeguard against XSS. As shown in Listing 10, termination is enforced when a failed nonce check triggers `wp_die`, halting execution before reaching sensitive operations. Alternatively, validation can store the verification result in a variable (e.g., `$nonce_verified`) that influences subsequent control flow, ensuring potentially unsafe paths are blocked. Together, these techniques strengthen defense by integrating conditional checks directly into execution paths.

V. Discussion

PHP dynamic Features: As a weakly typed scripting language, PHP possesses numerous complex dynamic semantics frequently employed by web applications and their plugins [6], [7], [29]. PSSA models many PHP features that influence static analysis; however, some features remain unsupported in a static context. PHP features such as dynamic arrays, dynamic code, and dynamic invocation are extensively used in CMS and web frameworks like Laravel [30]. Supporting these dynamic features during static analysis poses challenges, potentially resulting in overlooked vulnerabilities. The dynamic invocation issue is widespread across various languages, including reflection in Java [31]. Nevertheless, our experiments indicate that PSSA seldom produces false negatives due to these dynamic features. Consequently, we assert that PSSA effectively detects plugin vulnerabilities with reduced false positives, and supporting these dynamic features constitutes promising future work.

Path-sensitive security measures: As discussed earlier, our vulnerability-oriented path-sensitive analysis focuses exclusively on collecting constraints derived from user inputs or function parameters, deliberately ignoring those from global variables, system configurations, and similar sources. While this targeted approach improves efficiency, it may lead to occasional false positives due to untracked contextual factors. Additionally, simplifications made during constraint propagation and merging can introduce minor inaccuracies. To maximize coverage, we analyze the entire plugin codebase; however, file inclusion semantics may affect the order of analysis and variable scope, potentially contributing to imprecision. Despite these trade-offs, our empirical results demonstrate that most input-related constraints in web applications are relatively simple. Thus, prudent simplification not only maintains high analysis efficiency but also ensures practical accuracy, with only a small number of false positives observed during evaluation.

Automated Exploit Generation: Despite efforts to reduce false positives, some imprecision is inherent to static analysis. Integrating automated vulnerability verification and exploit generation therefore represents a promising direction for improving both precision and scalability. Prior work has demonstrated the effectiveness of symbolic execution–based techniques for automated exploit generation, significantly reducing manual verification effort and improving coverage [12], [32], [33]. Recent studies further highlight the growing role of intelligent and automated security analysis across diverse domains, including IoT firmware, cyber-physical systems, and insider threat detection [34], [35], [36]. Motivated by these advances and the emerging use of large language models in security analysis [37], [38], we plan to develop an independent exploit generation component that combines symbolic execution with AI-based reasoning to complement PSSA.

Support for other CMS plugins: Our system focuses on WordPress plugins due to WordPress's dominant role in the CMS ecosystem and the extensive prior research targeting it [1], [8], [39]. The CMS Knowledge Collector is, however, designed as a generic two-stage workflow that applies to any plugin-based PHP CMS, not as a WordPress-specific artefact. The static stage analyses the CMS source code with the same PHP analyser used for plugins, so it directly transfers to other PHP-based platforms; what needs to be added for a new CMS is the dynamic stage, which relies on injecting our collector *as a plugin* of the target CMS to observe global objects and argument-function behaviour from inside the running framework. For Joomla we would implement the collector as a Joomla plugin that subscribes to its event-dispatcher lifecycle; for Drupal, as a custom module that registers against Drupal's hook system. Once such a collector plugin is in place, the rest of PSSA — the static analyser and the path-sensitive vulnerability detector — can be applied to that CMS's plugin ecosystem unchanged. We view this generality as a property of the methodology rather than of any specific configuration, and we leave the engineering work of building dedicated Joomla and Drupal collectors as future work.

Particularly, the reported false positive rates, 28% for SQL injection and 33% for XSS, may seem high, even though they are much lower than those of tools like RIPS and phpSAFE. This is mainly due to the challenges of static analysis in PHP, a dynamic language with features like runtime type changes, dynamic includes, and reflection. These features make it hard to fully understand the code without actually running it. PSSA already uses several advanced techniques, e.g., type inference for object-oriented code, CMS-aware modeling, and path-sensitive analysis, to reduce false positives. However, some still occur, especially when dealing with dynamic strings across functions or indirect calls to sensitive functions. For path-sensitive analysis, PSSA focuses on constraints from user inputs and function parameters, which are the most relevant for real-world attacks. Including global variables or configuration values could make the analysis more complete, but also risks adding a lot of noise, since their meaning and behavior vary widely across plugins. To keep a good balance between accuracy and coverage, we choose

not to include them. Future work could explore smarter ways to selectively model these values when they are security-relevant.

VI. Related Work

Vulnerability detection in PHP applications: Previous studies [6], [7], [11], [18], [40], [41], [42], [43], [44], [45], [46], [47] have explored static analysis techniques to detect taint-style vulnerabilities such as XSS and SQL injection. Xie [11] and RIPS [6] introduced summary-based interprocedural taint analysis, while Pixy [18] focused on modeling aliasing, which, based on later findings [6], has limited impact in PHP analysis. Saner [40] further extended Pixy by incorporating dynamic modeling of user-defined sanitization. PHPJoern [48] leveraged code property graphs for interprocedural analysis, and TChecker [7] built upon it to handle object-oriented features in PHP. Other works have addressed different vulnerability types [10], [22], [33], [47], [49], [50], [51], [52], [53] and exploit generation [12], [32]. For example, FUSE [49] applies fuzzing to detect file upload vulnerabilities, LChecker [22] identifies loose comparison bugs, SSRFuzz [47] targets SSRF issues through combined analysis, and FUGIO [10] supports both detection and exploit generation for PHP object injection.

Analysis for CMS plugins: The security of CMS plugins has attracted increasing attention in recent years [1], [8], [9], [39], [50], [51], [54], [55]. YODA [1] was proposed as a framework for detecting and tracing malicious WordPress plugins, leveraging the analysis of over 40 K WordPress site backups to study malicious behavior and its impact on the plugin ecosystem. phpSAFE [9] focuses on object-oriented plugin vulnerability analysis but lacks type inference and relies solely on method names for call recognition; moreover, it requires manual configuration of WordPress source and sink functions, limiting scalability. CHKPLUG [8] addresses compliance by automatically verifying GDPR adherence in WordPress plugins, while Argus [55] identifies injection sinks and enhances existing SAST tools to uncover more vulnerabilities in plugins. Additionally, tools such as UChecker and UFuzzer [50], [51] explore unrestricted file upload vulnerabilities in WordPress environments. These studies collectively highlight the critical need for continued research focused on systematically uncovering security risks in CMS plugins.

Path-sensitive analysis for PHP applications: Symbolic execution has been widely used for vulnerability detection and exploit generation in PHP applications [12], [19], [41], [47], [50], [56]. Because PHP cannot be fully or accurately translated into C or LLVM IR [57], most approaches perform symbolic execution directly on PHP ASTs or CFGs and encode PHP-specific constraints into SMT formulas [12], [50], [58]. Prior work applies these techniques to different vulnerability classes, such as SSRFuzz for SSRF [47], Zheng et al. for RCE [19], and Navex for exploit generation via z3-str [12], [59], while systems like XSym and UChecker improve precision through built-in function modeling and vulnerability-oriented locality. Recent lightweight approaches such as Artemis [60] reduce false positives for specific vulnerabilities but rely on simplified path-sensitive modeling that does not generalize to more complex security checks.

VII. Conclusion

In this work, we introduce PSSA, a cutting-edge system designed for identifying vulnerabilities in plugins of PHP-based CMS. PSSA enhances the accuracy of plugin analysis by incorporating contextual information from the CMS code itself. By utilizing type inference for the analysis of PHP's object-oriented code and executing a vulnerability-focused, path-sensitive analysis, PSSA significantly reduces false positives. Through rigorous testing, PSSA has demonstrated superior performance compared to existing tools, uncovering a greater number of vulnerabilities while minimizing incorrect detections. Notably, PSSA has uncovered 178 new vulnerabilities, leading to the assignment of 82 CVE identifiers. These achievements establish PSSA as a leading solution in the field of CMS plugin vulnerability detection.

References

[1] R. P. Kasturi et al., "Mistrust plugins you must: A large-scale study of malicious plugins in WordPress marketplaces," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 161–178.

[2] O. Mesa et al., "Understanding vulnerabilities in plugin-based web systems: An exploratory study of wordpress," in *Proc. 22nd Int. Syst. Softw. Product Line Conf.*, New York, NY, USA, 2018, vol. 1, pp. 149–159,doi: 10.1145/3233027.3233042.

[3] "Usage statistics of content management systems," 2024. [Online]. Available: https://w3techs.com/technologies/overview/content_management

[4] "Wordpress plugins| wordpress.org," 2024. [Online]. Available: https://wordpress.org/plugins/

[5] "Wordpress vulnerability statistics," https://wpscan.com/statistics, 2024. [Online]. Available: https://wpscan.com/statistics

[6] J. Dahse and T. Holz, "Simulation of built-in php features for precise static code analysis," in *Proc. Netw. Distrib*. Syst. Secur. Symp., 2014, pp. 1–15.

[7] C. Luo, P. Li, and W. Meng, "TChecker: Precise static inter-procedural analysis for detecting taint-style vulnerabilities in php applications," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2022, pp. 2175–2188.

[8] F. H. Shezan et al., "CHKPLUG: Checking GDPR Compliance of WordPress Plugins Via Cross-Language Code Property Graph," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–17.

[9] P. J. C. Nunes, J. Fonseca, and M. Vieira, "phpSAFE: A security analysis tool for OOP web application plugins," in *Proc. Int. Conf. Dependable Syst. Netw.*, 2015, pp. 299–306.

[10] S. Park, D. Kim, S. Jana, and S. Son, "FUGIO: Automatic exploit generation for PHP object injection vulnerabilities," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 197–214.

[11] Y. Xie and A. Aiken, "Static detection of security vulnerabilities in scripting languages," in *Proc. USENIX Secur. Symp.*, 2006, pp. 179–192.

[12] A. Alhuzali, R. Gjomemo, B. Eshete, and V. Venkatakrishnan, "{NAVEX}: Precise and scalable exploit generation for dynamic web applications," in *Proc. USENIX Secur. Symp.*, 2018, pp. 377–392.

[13] J. Dahse and T. Holz, "Experience report: An empirical study of php security mechanism usage," in *Proc. Int. Symp. Softw. Testing Anal.*, 2015, pp. 60–70.

[14] "ThemeForest: Web themes & templates," 2024. [Online]. Available: https://themeforest.net/

[15] "Usage statistics of server-side programming languages for websites," 2024. [Online]. Available: https://w3techs.com/technologies/overview/programming_language

[16] "Wordpress security," https://developer.wordpress.org/apis/security/, 2024. [Online]. Available: https://developer.wordpress.org/apis/security/

[17] A. Milanova, A. Rountev, and B. G. Ryder, "Parameterized object sensitivity for points-to analysis for java," *ACM Trans. Softw. Eng. Methodol.*, vol. 14, no. 1, pp. 1–41, 2005.

[18] N. Jovanovic, C. Kruegel, and E. Kirda, "Pixy: A static analysis tool for detecting web application vulnerabilities," in *Proc. IEEE Symp. Secur. Privacy*, 2006, pp. 258–263.

[19] Y. Zheng and X. Zhang, "Path sensitive static analysis of web applications for remote code execution vulnerability detection," in *Proc. 3th Int. Conf. Softw. Eng.*, 2013, pp. 652–661.

[20] "Nikic/php-parser: A php parser written in php," 2024. [Online]. Available: https://github.com/nikic/PHP-Parser

[21] "Ircmaxell/php-cfg: A control flow graph implementation in php," , 2024. [Online]. Available: https://github.com/ircmaxell/php-cfg

[22] P. Li and W. Meng, "LChecker: Detecting loose comparison bugs in PHP," in *Proc. Web Conf.*, 2021, pp. 2721–2732.

[23] Y. Shi et al., "{XSSky }: Detecting { XSS} vulnerabilities through local {Path-Persistent} fuzzing," in *Proc 34th USENIX Secur. Symp.* 25, 2025, pp. 8255–8272.

[24] "Wordpress popular plugins,"2024. [Online]. Available: https://wordpress.org/plugins/browse/popular/

[25] "MC4WP: Mailchimp for WordPress," 2022. [Online]. Available: https://cn.wordpress.org/plugins/mailchimp-for-wp/advanced/

[26] "Wpscan: Wordpress security," 2024. [Online]. Available: https://wpscan.com/

[27] "Ninja forms contact form–the drag and drop form builder for wordpress – wordpress plugin |wordpress.org," 2024. [Online]. Available: https://wordpress.org/plugins/ninja-forms/

[28] "Paid membership plugin, ecommerce, registration form, login form, user profile & restrict content - profilepress - wordpress plugin| wordpress.org," 2024. [Online]. Available: https://wordpress.org/plugins/wp-user-avatar/

[29] M. Hills, P. Klint, and J. Vinju, "An empirical study of PHP feature usage: A static analysis perspective," in *Proc. Int. Symp. Softw. Testing Anal.*, 2013, pp. 325–335.

[30] "Laravel - the PHP framework for web artisans," 2024. [Online]. Available: https://laravel.com/

[31] D. Landman, A. Serebrenik, and J. J. Vinju, "Challenges for static analysis of java reflection - literature review and empirical study," in *Proc. IEEE/ACM 39th Int. Conf. Softw. Eng.*, 2017, pp. 507–518.

[32] A. Alhuzali, B. Eshete, R. Gjomemo, and V. Venkatakrishnan, "Chainsaw: Chained automated workflow-based exploit generation," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 641–652.

[33] O. Olivo, I. Dillig, and C. Lin, "Detecting and exploiting second order denial-of-service vulnerabilities in web applications," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2015, pp. 641–652. .

[34] X. Feng, X. Zhu, Q.-L. Han, W. Zhou, S. Wen, and Y. Xiang, "Detecting vulnerability on IoT device firmware: A survey," *IEEE/CAA J. Automatica Sinica*, vol. 10, no. 1, pp. 25–41, Jan. 2022.

[35] J. Zhang, L. Pan, Q.-L. Han, C. Chen, S. Wen, and Y. Xiang, "Deep learning based attack detection for cyber-physical system cybersecurity: A survey," *IEEE/CAA J. Automatica Sinica*, vol. 9, no. 3, pp. 377–391, Mar. 2021.

[36] L. Liu, O. De Vel, Q.-L. Han, J. Zhang, and Y. Xiang, "Detecting and Preventing Cyber Insider Threats: A Survey," *IEEE Commun. Surveys Tuts*, vol. 20, no. 2, pp. 1397–1417, Secondquarter 2018.

[37] Z. Deng et al., "Exploring deepseek: A survey on advances, applications, challenges and future directions," *IEEE/CAA J. Automatica Sinica*, vol. 12, no. 5, pp. 872–893, May 2025.

[38] X. Zhu, W. Zhou, Q.-L. Han, W. Ma, S. Wen, and Y. Xiang, "When software security meets large language models: A survey," *IEEE/CAA J. Automatica Sinica*, vol. 12, no. 2, pp. 317–334, Feb. 2025.

[39] J. Lin, M. Sayagh, and A. E. Hassan, "The co-evolution of the wordpress platform and its plugins," *ACM Trans. Softw. Eng. Methodol.*, pp. 1–24, 2023.

[40] D. Balzarotti et al., "Saner: Composing static and dynamic analysis to validate sanitization in web applications," in *Proc. IEEE Symp. Secur. Privacy*, 2008, pp. 387–401.

[41] S. Son and V. Shmatikov, "SAFERPHP: Finding semantic vulnerabilities in PHP applications," in *Proc. ACM SIGPLAN 6th Workshop Program. Lang. Anal. Secur.*, 2011, pp. 1–13.

[42] M. K. Gupta, M. C. Govil, and G. Singh, "Predicting cross-site scripting (xss) security vulnerabilities in web applications," in *Proc. 12th Int. Joint Conf. Comput. Sci. Softw. Eng.*, 2015, pp. 162–167.

[43] I. Medeiros, N. Neves, and M. Correia, "DEKANT: A static analysis tool that learns to detect web application vulnerabilities," in *Proc. 25th Int. Symp. Softw. Testing Anal.*, 2016, pp. 1–11.

[44] M. Sridharan, S. Artzi, M. Pistoia, S. Guarnieri, O. Tripp, and R. Berg, "F4F: Taint analysis of framework-based web applications," in *Proc. ACM Int. Conf. Object Oriented Program. Syst. Lang. Appli.*, 2011, pp. 1053–1068.

[45] S. Wi, S. Woo, J. J. Whang, and S. Son, "HiddenCPG: Large-scale vulnerable clone detection using subgraph isomorphism of code property graphs," in *Proc. ACM Web Conf.*, 2022, pp. 755–766.

[46] Y. Huang, C. Shi, J. Lu, H. Li, H. Meng, and L. Li, "Detecting broken object-level authorization vulnerabilities in database-backed applications," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2024, pp. 2934–2948.

[47] E. Wang et al., "Where URLs become weapons: Automated discovery of SSRF vulnerabilities in web applications," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 239–257.

[48] M. Backes, K. Rieck, M. Skoruppa, B. Stock, and F. Yamaguchi, "Efficient and flexible discovery of PHP application vulnerabilities," in *Proc. IEEE Eur. Symp. Secur. Privacy*, 2017, pp. 334–349.

[49] T. Lee, S. Wi, S. Lee, and S. Son, "FUSE: Finding file upload bugs via penetration testing," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–17.

[50] J. Huang, Y. Li, J. Zhang, and R. Dai, "UChecker: Automatically detecting PHP-based unrestricted file upload vulnerabilities," in *Proc. 49th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, 2019, pp. 581–592.

[51] J. Huang, J. Zhang, J. Liu, C. Li, and R. Dai, "UFuzzer: Lightweight detection of PHP-Based unrestricted file upload vulnerabilities via static-fuzzing co-analysis," in *Proc. 24th Int. Symp. Res. Attacks, Intrusions Defenses*, 2021, pp. 78–90.

[52] J. Dahse and T. Holz, "Static detection of second-order vulnerabilities in web applications," in *Proc. 23rd USENIX Secur. Symp.*, 2014, pp. 989–1003.

[53] A. Naderi-Afooshteh, Y. Kwon, A. Nguyen-Tuong, A. Razmjoo-Qalaei, M.-R. Zamiri-Gourabi, and J. W. Davidson, "Malmax: Multi-aspect execution for automated dynamic web server malware analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1849–1866.

[54] D. T. Murphy, M. F. Zibran, and F. Z. Eishita, "Plugins to detect vulnerable plugins: An empirical assessment of the security scanner plugins for wordpress," in *Proc. IEEE/ACIS 19th Int. Conf. Softw. Eng. Res. Manage. Appli.*, 2021, pp. 39–44.

[55] R. Jahanshahi and M. Egele, "Argus: All your php injection-sinks are belong to us," in *Proc. 33rd USENIX Secur.*, 2024, pp. 6759–6776.

[56] R. Baldoni, E. Coppa, D.C. D'elia, C. Demetrescu, and I. Finocchi, "A survey of symbolic execution techniques," *ACM Comput. Surv.*, vol. 51, no. 3, pp. 1–39, 2018.

[57] P. Li, W. Meng, K. Lu, and C. Luo, "On the feasibility of automated built-in function modeling for PHP symbolic execution," in *Proc. Web Conf.*, 2021, pp. 58–69.

[58] P. Li, W. Meng, M. Zhang, C. Wang, and C. Luo, "Holistic concolic execution for dynamic web applications via symbolic interpreter analysis," in *Proc. IEEE Symp. Secur. Privacy*, 2024, pp. 222–238.

[59] Y. Zheng, X. Zhang, and V. Ganesh, "Z3-Str: A z3-based string solver for web application analysis," in *Proc. 9th Joint Meeting Foundations Softw. Eng.*, 2013, pp. 114–124.

[60] Y. Ji, T. Dai, Z. Zhou, Y. Tang, and J. He, "Artemis: Toward accurate detection of server-side request forgeries through LLM-assisted inter-procedural path-sensitive taint analysis," in *Proc. ACM Program. Lang.*, vol. 9, no.OOPSLA1, pp. 1301349–1377, 2025.